오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

HTML/CSS

DAY 06

반응형 웹과 HTML/CSS 종합 실습

화면 크기가 달라져도 읽기 좋은 페이지를 만듭니다.
지금까지 만든 개인 소개 페이지를 점검하고 AI 리뷰로 다듬어 완성합니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **화면 크기에 따라 스타일을 바꿀 수 있다**
   미디어 쿼리와 상대 단위로 모바일 퍼스트 화면을 만듭니다.

2. **내 페이지의 오류를 스스로 점검할 수 있다**
   레이아웃, 선택자 충돌, 시맨틱과 접근성을 순서대로 확인합니다.

3. **AI 리뷰를 검증해 반영할 수 있다**
   근거를 확인한 제안만 골라 개인 소개 페이지를 완성합니다.

오늘의 결과물

세 폭에서 자연스럽게 보이는 개인 소개 페이지 최종본

ROADMAP

오늘의 흐름

01

반응형이란

필요한 이유
viewport

02

상대 단위

%와 vw·vh
rem과 em

03

미디어 쿼리

@media 조건
실습 01

실습 포함

04

모바일 퍼스트

분기점 정하기
실습 02

실습 포함

05

종합 점검

AI 리뷰
실습 03

실습 포함

단위를 먼저 바꿔두면 미디어 쿼리로 손볼 곳이 크게 줄어듭니다.
마지막에 지금까지 만든 페이지를 점검하고 AI 리뷰를 받아 1챕터를 마무리합니다.

되짚기 · 01

닷새 동안 만든 것

1	**DAY 01~03 · 구조**	문서 구조와 폼, 시맨틱 태그로 뼈대를 만들었습니다
2	**DAY 04 · 선택자**	style.css를 연결하고 원하는 요소를 골랐습니다
3	**DAY 05 · 레이아웃**	박스 모델과 Flexbox, Grid로 배치했습니다
4	**지금 상태**	화면을 좁히면 카드가 같이 좁아지기만 합니다

오늘은 화면 크기에 따라 배치 자체가 바뀌게 만듭니다

되짚기 · 02

카드 세 개가 좁은 화면에서는 읽기 어렵습니다

```
.cards {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}

/* 화면이 390px이어도 계속 세 열입니다 */
```

무엇이 문제인가

한 칸이 100px 남짓이 되어
글자가 두세 자씩 끊깁니다.

오늘 배우는 것

화면이 좁을 때는 한 열,
넓을 때는 세 열이 되도록
조건을 나누는 방법입니다.

준비

오늘의 셋팅

```
Day06/
  about-me.html    ← Day05 파일 그대로 씁니다
  style.css        ← Day05 파일을 이어서 고칩니다
  me.jpg

/* 오늘은 HTML을 거의 건드리지 않습니다 */
/* 같은 HTML에 CSS만으로 화면이 달라집니다 */
```

개발자 도구 기기 모드

- Mac은 ⌘⇧M, Windows는 Ctrl+Shift+M
- 화면 폭이 숫자로 보입니다
- 기기를 골라 볼 수 있습니다

오늘 계속 볼 숫자

왼쪽 위에 표시되는 폭입니다.
분기점을 눈으로 확인합니다.

01

반응형이란 무엇인가

필요한 이유 · viewport · 픽셀과 해상도

01 · 배경

같은 페이지를 다양한 크기의 화면에서 봅니다

휴대폰

360 ~ 440px

한 손으로 세로로 봅니다.
한 줄에 들어가는 글자가
많지 않습니다.

태블릿

740 ~ 1040px

가로로 돌리면 폭이
1.3~1.5배로 넓어집니다.

데스크톱

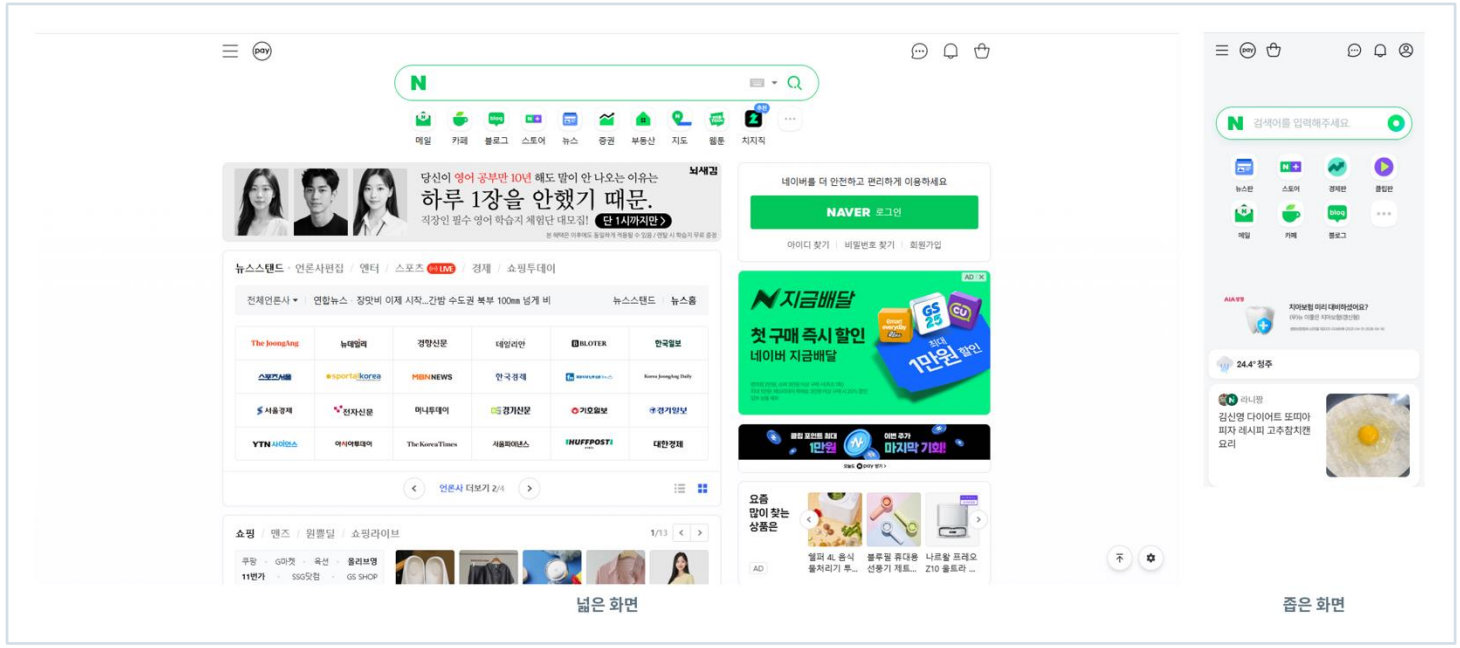
1280px 이상

창 크기를 사용자가
마음대로 바꿉니다.
폭이 고정되어 있지 않습니다.

화면마다 다른 페이지를 만드는 것이 아닙니다

하나의 HTML과 CSS로 여러 화면에 대응하는 것이 반응형 웹 디자인입니다.

같은 사이트를 두 폭에서 열어 봅니다



왼쪽은 넓은 화면, 오른쪽은 좁은 화면에서 같은 사이트를 연 모습

주소는 같은데 배치가 다릅니다

01 · 정의

반응형 웹 디자인

여러 기기와 화면 크기에 맞춰 페이지의 배치와 내용을 조정하는 설계 방식입니다.

유연한 레이아웃

px 대신 %, vw, rem 같은
상대 단위를 씁니다.
폭이 달라지면 함께 달라집니다.

미디어 쿼리

화면 크기나 기기 특성에 따라
다른 규칙을 적용합니다.
오늘의 중심입니다.

반응형 이미지

이미지가 화면을 넘지 않게
하고, 필요하면 화면에 맞는
파일을 따로 내려받습니다.

하나의 코드로 모든 화면에 대응할 수 있습니다

오늘은 이 세 가지를 순서대로 다룹니다.

01 · 필요성

왜 반응형으로 만드나요

사용자

읽을 수 있어야 합니다

웹 접속의 절반 이상이
휴대폰에서 일어납니다.
좁은 화면에서 확대하고 좌우로
밀어야 한다면 대부분 떠납니다.

만드는 쪽

하나만 관리하면 됩니다

모바일용 사이트를 따로 만들면
고칠 곳이 두 배가 됩니다.
주소도 하나로 유지되어
검색에도 유리합니다.

구글은 모바일 화면을 기준으로 페이지를 색인합니다

좁은 화면에서 제대로 보이지 않으면 검색 결과에서도 불리해집니다.

이 한 줄이 없으면 분기점이 엉뚱하게 잡힙니다

```
<head>
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
</head>

/* width=device-width : 기기 폭을 그대로 씁니다 */
/* initial-scale=1.0  : 처음 배율을 1로 둡니다 */
```

없으면 어떻게 되나요

휴대폰이 데스크톱 폭이라고
가정하고 축소해서 보여줍니다.
글자가 아주 작아집니다.

CSS 픽셀과 기기 픽셀은 다릅니다

CSS 픽셀

우리가 적는 숫자

width: 100px 이라고 적을 때의
그 100입니다.
논리적인 단위라서 기기가 달라도
비슷한 크기로 보입니다.

기기 픽셀

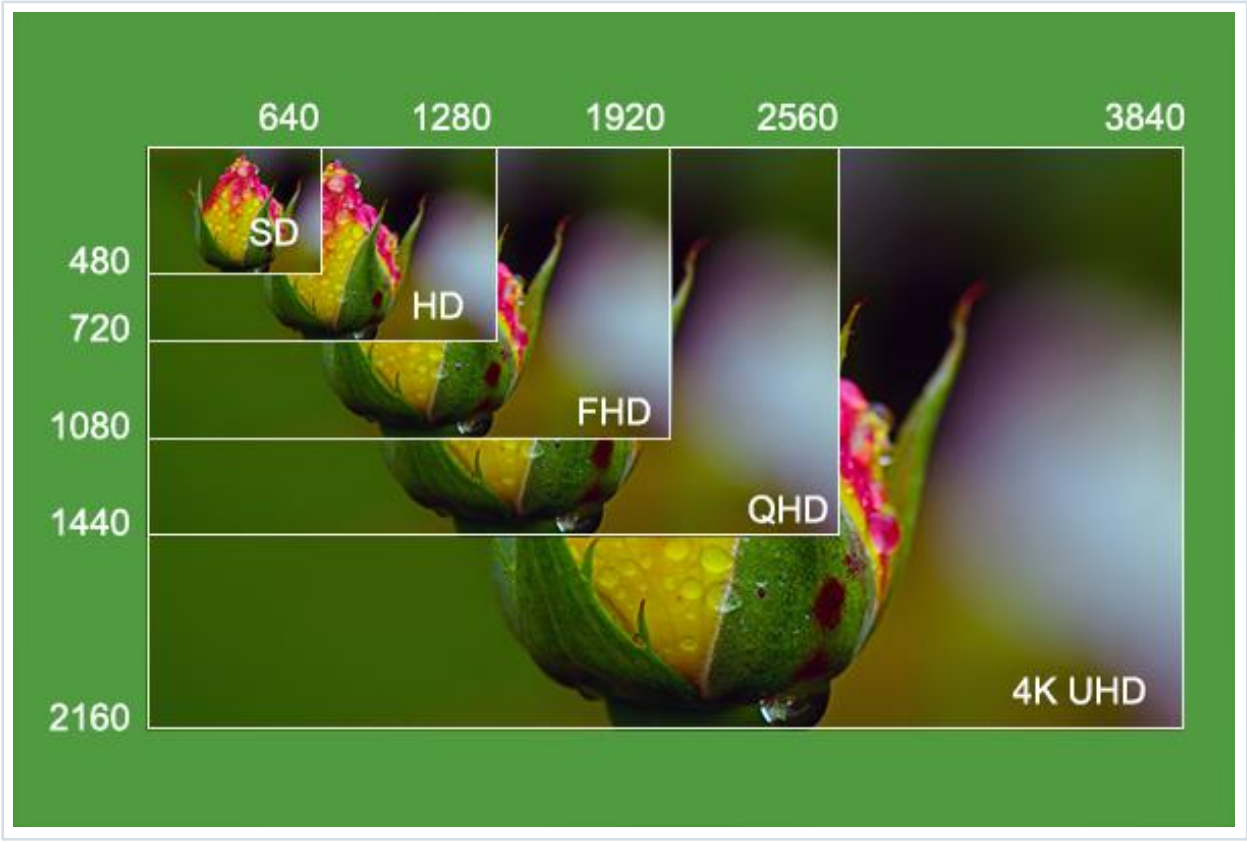
화면의 실제 점

고해상도 화면은 같은 자리에
점을 더 촘촘히 찍습니다.
그 비율을 DPR이라고 부릅니다.

DPR이 2인 화면에서 가로 100px은 실제로 점 200개로 그려집니다

그래서 이미지는 화면에 보이는 크기보다 큰 파일을 준비해야 선명하게 보입니다.

해상도는 화면이 가진 점의 개수입니다



같은 사진을 해상도별로 겹쳐 놓은 그림

해상도는 점의 개수

1920×1080처럼 화면이
가로세로로 몇 개의 점을
가졌는지를 말합니다.

미디어 쿼리는 다릅니다

@media의 768px은 해상도가
아니라 CSS 픽셀 기준의
뷰포트 폭입니다.

01 · 분기점

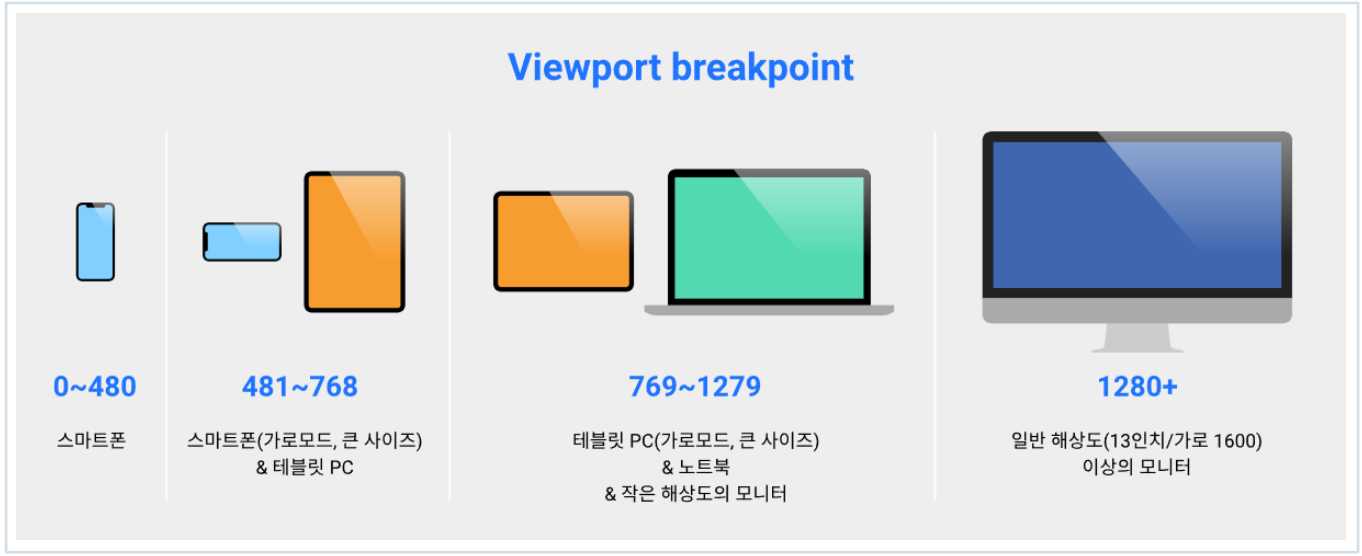
많이 쓰는 폭 구간

1	**320 ~ 480px**	휴대폰 세로. 한 열로 쌓는 구간입니다
2	**768px**	태블릿 세로. 두 열로 나누기 시작하는 지점입니다
3	**1024px**	태블릿 가로와 작은 노트북. 세 열이 들어갑니다
4	**1280px 이상**	데스크톱. 폭을 더 늘리기보다 최대 폭을 정해둡니다

숫자를 외우지 말고 내 콘텐츠가 깨지는 지점을 찾습니다

창을 천천히 좁히다가 글자가 어색해지는 폭이 그 페이지의 분기점입니다.

흔히 쓰는 구간을 그림으로 보면



널리 쓰이는 구간의 예시입니다. 표준으로 정해진 값은 아닙니다.

이 숫자를 외우는 것이 목적이 아닙니다

내 페이지가 깨지는 폭을 찾아 그 자리에 분기점을 두는 것이 먼저입니다.

여기까지 정리

1	**반응형**	하나의 HTML과 CSS로 여러 화면에 대응하는 방식입니다
2	**세 가지 재료**	상대 단위, 미디어 쿼리, 반응형 이미지
3	**viewport**	없으면 휴대폰이 약 980px 화면인 척해 분기점이 어긋납니다
4	**분기점**	기기 목록이 아니라 콘텐츠가 깨지는 지점으로 정합니다

다음은 px을 대신할 단위들입니다

02

상대 단위

% · vw와 vh · em과 rem

무엇을 기준으로 삼는 단위인가

단위	기준	쓰는 곳
px	고정값, 기준이 없습니다	테두리 굵기처럼 변하면 안 되는 값
%	부모 요소의 크기	너비, 여백
vw	화면 가로의 1%	화면 전체를 쓰는 영역
vh	화면 세로의 1%	한 화면을 꽉 채우는 섹션
em	그 요소의 글자 크기	글자에 비례하는 여백
rem	html의 글자 크기	글자 크기, 여백, 폭 전반

실무에서는 rem을 기본으로 쓰고 나머지를 필요할 때 섞습니다

02 · %

% — 부모의 크기를 기준으로

```
.parent {
  width: 500px;
}
.child {
  width: 50%;     /* 250px */
}

/* 부모가 달라지면 자식도 함께 달라집니다 */
```

부모가 기준입니다

화면이 아니라 바로 위
요소의 크기를 기준으로 합니다.

높이에는 주의

height: 50%는 부모의 높이가
정해져 있어야 동작합니다.

02 · vw

vw — 화면 가로의 1%

```
.hero {
  width: 100vw;     /* 스크롤바까지 포함한 화면 전체 */
  height: 50vw;     /* 가로의 절반 높이 */
}

.title {
  font-size: 4vw;   /* 화면에 따라 글자도 커집니다 */
}
```

부모와 상관없습니다

화면 폭만 봅니다.
부모가 좁아도 100vw는
화면 전체 폭입니다.

글자에 쓸 때는 조심

아주 좁은 화면에서 글자가
너무 작아질 수 있습니다.

02 · vh

vh — 화면 세로의 1%

```
.section {
  min-height: 100vh;   /* 적어도 한 화면을 채웁니다 */
  display: flex;
  align-items: center;
  justify-content: center;
}

/* 첫 화면을 한 장으로 만들 때 씁니다 */
```

쓰는 곳

첫 화면을 한 장으로 채우는
랜딩 페이지에 자주 씁니다.

모바일에서는 어긋납니다

주소창이 접히고 펴지면서
실제 높이가 달라집니다.
화면이 살짝 잘려 보입니다.

em — 그 요소의 글자 크기를 기준으로

```
.parent { font-size: 16px; }

.child {
  font-size: 1.5em;    /* 16 × 1.5 = 24px */
  padding: 0.5em;      /* 24 × 0.5 = 12px */
}
```

기준이 두 번 바뀝니다

font-size에 쓰면 부모의 글자,
다른 속성에 쓰면 자기 자신의
글자 크기가 기준입니다.

겹치면 계산이 어려워집니다

em을 쓴 요소 안에 또 em을 쓰면
값이 곱해집니다.

rem — html의 글자 크기를 기준으로

```
/* html의 기본 글자 크기는 보통 16px입니다 */

.card {
  font-size: 1.5rem;    /* 24px */
  padding: 1rem;        /* 16px */
  gap: 0.5rem;          /* 8px */
}
```

기준이 하나뿐입니다

어디에 있든 html의 글자 크기만 봅니다. 계산이 예측됩니다.

접근성에도 좋습니다

사용자가 브라우저 글자 크기를 키우면 rem 값도 함께 커집니다. html에 px로 고정하면 이 설정이 무시됩니다.

02 · 적용

고정 단위를 상대 단위로 바꿔봅니다

이전

px으로 고정

```
.card {
  width: 320px;
  padding: 20px;
}
```

이후

상대 단위로

```
.card {
  width: 100%;
  max-width: 20rem;
  padding: 1.25rem;
}
```

좁으면 폭이 줄고 넓어지면 20rem에서 멈춥니다

03

미디어 쿼리

@media 조건 · 화면 크기 · 실습 01

조건을 만족할 때만 적용되는 규칙

```
@media (min-width: 768px) {
  .cards {
    grid-template-columns: 1fr 1fr;
  }
}

/* 화면이 768px 이상일 때만
   안쪽 규칙이 적용됩니다 */
```

평소 쓰던 규칙을 감쌉니다

새로운 문법을 배우는 것이
아니라 이미 아는 규칙을
조건 안에 넣는 것입니다.

중괄호가 두 겹입니다

닫는 괄호를 빠뜨리는 실수가
가장 많이 나옵니다.

무엇을 대상으로 하는 조건인가

1	**screen**	화면에 보여주는 기기. 실제로 거의 이 값만 씁니다
2	**print**	인쇄할 때. 배경색을 빼거나 링크 주소를 펼칠 때 씁니다
3	**all**	모든 기기. 유형을 적지 않으면 이 값으로 동작합니다

유형은 생략해도 됩니다

@media (min-width: 768px) 처럼 조건만 적으면 all과 같습니다. 실무에서는 대부분 이렇게 씁니다.

min-width와 max-width

```
@media (max-width: 767px) { ... }
/* 767px 이하일 때 */

@media (min-width: 768px) { ... }
/* 768px 이상일 때 */

@media (min-width: 768px) and (max-width: 1023px) { ... }
/* 768px 이상 1023px 이하, 즉 그 사이 구간 */
```

min은 이상, max는 이하

min-width는 그 값과 같거나
넓을 때 켜집니다.
이름과 방향이 반대로
느껴지니 주의합니다.

and로 두 조건을 이으면 구간이 됩니다

겹치지 않게 끊습니다. 폭이 소수로 나오는 기기가 있어 767.98px처럼 적기도 합니다.

orientation — 가로 모드와 세로 모드

```
@media (orientation: landscape) {
  .container { flex-direction: row; }
}

@media (orientation: portrait) {
  .container { flex-direction: column; }
}
```

가로가 길면 landscape

세로가 더 길면 portrait입니다.
휴대폰을 돌리면 바뀝니다.

폭 조건으로 충분합니다

방향을 직접 보기보다
min-width로 처리하는 편이
관리하기 쉽습니다.

min-resolution — 고해상도 화면 대응

```
@media (min-resolution: 2dppx) {
  .logo {
    background-image: url('logo@2x.png');
  }
}

/* dppx는 CSS 픽셀 하나에 점이 몇 개인지 */
```

앞에서 본 DPR과 같은 이야기

2dppx는 DPR이 2인 화면입니다.
가로세로 두 배씩, 면적으로는
네 배의 점이 찍힙니다.

주로 이미지에 씁니다

레이아웃을 바꾸는 데는
쓰지 않습니다.

prefers-color-scheme — 설정을 따라갑니다

```
@media (prefers-color-scheme: dark) {
  body {
    background-color: #1f2e3a;
    color: #e6eef5;
  }
}

/* 운영체제의 다크 모드 설정을 읽습니다 */
```

화면 크기가 아닌 조건

미디어 쿼리는 폭만 보는 것이
아니라 사용자의 환경도
조건으로 삼을 수 있습니다.

확인 방법

Mac은 시스템 설정에서
바로 바꿔볼 수 있습니다.

and와 쉼표로 조건을 엮습니다

```
@media (min-width: 768px) and (max-width: 1023px)
/* 둘 다 만족할 때 */

@media (max-width: 480px), (orientation: portrait)
/* 둘 중 하나만 만족해도 */

@media not print
/* 인쇄가 아닐 때 */
```

쉼표가 or 입니다

and는 그리고,
쉼표는 또는이라고
읽으면 됩니다.

실무에서는 min-width 하나로 끝나는 경우가 대부분입니다

조건을 복잡하게 엮을수록 나중에 읽기 어려워집니다. 단순하게 유지하세요.

03 · 확인

이 속성을 지금 써도 되는지 확인합니다

CSS에는 최근에 추가된 속성이 계속 늘어납니다.
브라우저가 지원하는지 확인하고 쓰는 습관이 필요합니다.

참고 Can I use — 속성이 어느 브라우저에서 동작하는지 확인하는 사이트

이렇게 씁니다

- 속성 이름을 검색창에 적는다
- 초록색이면 지원, 빨간색이면 미지원
- 점유율 합계를 확인한다

오늘 배운 것은

미디어 쿼리와 상대 단위는
모든 최신 브라우저에서 동작합니다.
안심하고 써도 됩니다.

레퍼런스 MDN — 미디어 쿼리 사용하기

실습 01

미디어 쿼리 넣어보기

DAY 05에서 만든 카드 페이지를 열고 기본 스타일을 한 열로 바꿉니다.
그다음 분기점을 넣어 폭에 따라 열 수가 달라지게 만듭니다.

```
.cards {
  display: grid;
  grid-template-columns: 1fr;    /* 기본은 한 열 */
  gap: 1rem;
}
@media (min-width: 768px) {
  .cards { grid-template-columns: 1fr 1fr; }
}
```

확인하는 방법

- 기기 모드를 켠다
- 폭을 천천히 줄인다
- 열 수가 바뀌는 지점을 본다

폭 숫자를 보면서 진행합니다

767에서 768로 넘어가는 순간을 함께 봅니다.

그다음

1024px 분기를 더 넣습니다.

실습 01 · 점검

분기점에서 안 바뀐다면

01

중괄호 짝

@media의 닫는 괄호를
빠뜨리지 않았는지 봅니다.
모자라면 아래 규칙이 모두
그 조건 안으로 들어갑니다.

02

적은 순서

1024px 규칙을 768px보다
위에 적으면 나중에 적은
768px 규칙이 이깁니다.
좁은 것부터 적습니다.

03

min과 max

min-width는 그 값 이상,
max-width는 이하입니다.
반대로 적으면 규칙이
정반대로 동작합니다.

미디어 쿼리도 결국 CSS 규칙이라 DAY 04의 우선순위가 그대로 적용됩니다

04

모바일 퍼스트

설계 순서 · 반응형 이미지 · 실습 02

04 · 두 방식

어느 화면을 기본으로 두고 시작할까

모바일 퍼스트

좁은 화면이 기본

기본 스타일을 모바일에 맞추고
넓어질 때 규칙을 더합니다.

```
.cards { grid-template-columns: 1fr; }
@media (min-width: 768px) { ... }
```

데스크톱 퍼스트

넓은 화면이 기본

기본 스타일을 데스크톱에 맞추고
좁아질 때 규칙을 덜어냅니다.

```
.cards { grid-template-columns: 1fr 1fr 1fr; }
@media (max-width: 767px) { ... }
```

min-width로 쌓아 올리느냐, max-width로 깎아 내리느냐의 차이입니다

새로 만든다면 모바일퍼스트

01

단순한 것부터

좁은 화면은 한 열로 쌓기
때문에 규칙이 적습니다.
여기서 시작하면
복잡도가 천천히 올라갑니다.

02

빠뜨리지 않습니다

데스크톱부터 만들면
모바일 확인을 미루다가
마지막에 크게 고치는 일이
자주 생깁니다.

03

접속 비율

웹 접속의 절반 이상이
휴대폰에서 일어납니다.
가장 많은 사용자를
먼저 챙기는 순서입니다.

이 과정에서 만드는 모든 화면은 모바일 퍼스트로 만듭니다

목적에 따라 데스크톱 퍼스트

01

이미 있는 코드

오래된 사이트를 고칠 때는
기본 스타일이 이미
데스크톱 기준으로
적혀 있습니다.

02

업무용 화면

관리자 대시보드나
데이터가 많은 표는
넓은 화면에서 주로
쓰입니다.

03

선택의 문제

둘 중 무엇이 옳다기보다
무엇을 기본으로 둘지
정하고 일관되게
가는 것이 중요합니다.

한 파일 안에서 두 방식을 섞으면 규칙이 서로 덮어써 관리가 어려워집니다

04 · 순서

좁은 것부터 넓은 것 순서로 적습니다

```
/* 기본 — 모바일 */
.cards { grid-template-columns: 1fr; }

@media (min-width: 768px) {
  .cards { grid-template-columns: 1fr 1fr; }
}

@media (min-width: 1024px) {
  .cards { grid-template-columns: 1fr 1fr 1fr; }
}
```

순서가 곧 규칙입니다

1280px 화면에서는 세 조건이 모두 참이 됩니다. 마지막에 적은 규칙이 이겨서 세 열이 됩니다.

거꾸로 적으면

768px 규칙이 나중에 적혀 넓은 화면에서도 두 열이 되어 버립니다.

이미지가 화면을 넘지 않게 합니다

```
img {
  max-width: 100%;    /* 부모보다 커지지 않게 */
  height: auto;       /* 비율을 지키게 */
}

<!-- 화면에 따라 다른 파일을 쓰고 싶다면 -->
<img src="small.jpg"
     srcset="small.jpg 480w, large.jpg 1200w"
     sizes="(min-width: 768px) 50vw, 100vw"
     alt="작업 사진">
```

두 줄이면 대부분 해결됩니다

가로 스크롤이 생기는 원인의
대부분이 이미지입니다.

srcset은 다음 단계

sizes로 실제 표시 폭을 알려줘야
브라우저가 맞는 파일을 고릅니다.
오늘은 이런 방법이 있다는
정도만 알아둡니다.

내비게이션도 같은 방법으로 바꿉니다

```
/* 기본 — 위아래로 쌓입니다 */
.nav {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

@media (min-width: 768px) {
  .nav { flex-direction: row; justify-content: space-between; }
}
```

DAY 05 속성을 그대로

새로운 것을 배우는 것이
아니라 이미 아는 속성을
조건 안에 넣는 것입니다.

값 하나만 바꿉니다

flex-direction 하나로
세로와 가로가 바뀝니다.

실습 02

1열·2열·3열로 바뀌는 카드 완성하기

카드 페이지를 세 단계로 완성하고 내비게이션까지 반응형으로 바꿉니다.
이미지가 화면을 넘지 않도록 함께 손봅니다.

해야 할 것

- viewport 메타 태그를 확인한다
- 기본을 한 열로 만든다
- 768px과 1024px 분기를 넣는다
- 내비게이션 방향을 바꾼다
- 이미지에 max-width를 준다

확인할 것

- 가로 스크롤이 생기지 않는다
- 글자가 지나치게 작지 않다
- 세 구간에서 열 수가 맞다
- 카드 간격이 유지된다
- 휴대폰에서도 열어 확인한다

다 한 사람은 기기 모드에서 화면을 가로로 돌려 열 수가 어떻게 바뀌는지 확인합니다

점검

반응형에서 자주 나오는 세 가지

!

휴대폰에서 글자가 작아요

viewport 메타 태그가
빠진 경우가 대부분입니다.
head를 먼저 확인합니다.

!

가로 스크롤이 생겨요

이미지나 고정 폭 요소가
화면보다 넓은 경우입니다.
max-width를 확인합니다.

!

분기점이 너무 많아요

기기마다 조건을 만들면
관리가 어려워집니다.
두세 개로 줄입니다.

반응형은 화면마다 다른 페이지를 만드는 일이 아닙니다

기본을 좁은 화면에 맞춰 두고, 넓어질 때 필요한 규칙만 더하는 방식으로 생각하면 단순해집니다.

05

종합 점검과 AI 리뷰

레이아웃·선택자 점검 · AI 리뷰 · 실습 03

레이아웃과 반응형 오류 점검

좁은 화면

- 가로 스크롤이 없다
- 글자가 잘리지 않는다
- 버튼을 누를 수 있다

분기점 근처

- 열 수가 제때 바뀐다
- 간격이 갑자기 벌어지지 않는다
- 이미지 비율이 깨지지 않는다

넓은 화면

- 양옆이 지나치게 비지 않는다
- 글줄이 너무 길지 않다
- 카드 크기가 일정하다

오늘 만든 카드 페이지와 개인 소개 페이지를 세 폭에서 각각 확인합니다

개발자 도구의 기기 모드에서 폭을 바꿔가며 보고, 가능하면 자기 휴대폰으로도 열어봅니다.

선택자 충돌 점검

```
/* 같은 요소를 두 규칙이 고르고 있다면 */
.card { background: white; }
#intro .card { background: #eef; }

/* 뒤쪽이 더 구체적이라 이깁니다 */

/* Styles 탭에서 취소선이 그어진 규칙이
   진 쪽입니다 */
```

확인하는 순서

요소를 우클릭하고 검사,
Styles 탭의 취소선을 봅니다.

정리하면 좋은 것

쓰지 않는 규칙,
같은 요소를 두 번 고르는 규칙,
!important로 눌러둔 규칙

지울 때는 이 순서로

01

지우기 전에 확인

개발자 도구에서 그 규칙을
잠깐 꺼 봅니다.
화면이 그대로면
지워도 되는 규칙입니다.

02

한 번에 하나씩

여러 규칙을 동시에 지우면
무엇 때문에 깨졌는지
찾을 수 없습니다.

03

지운 뒤 다시 보기

세 폭에서 각각 확인합니다.
좁은 화면에서만 쓰이던
규칙일 수 있습니다.

지우고 나서 화면이 그대로인지 확인하는 것까지가 한 세트입니다

검사기는 이렇게 씁니다

직접 눈으로 찾기 어려운 구조 오류는 도구가 대신 찾아줍니다.

W3C 마크업 검사기

Direct input 칸에 HTML을
붙여 넣으면 오류와 경고를
줄 번호와 함께 알려줍니다.

WAVE

접근성 문제를 화면 위에
표시해 줍니다.
alt 누락과 대비 부족을
빠르게 찾습니다.

Lighthouse

크롬 개발자 도구에
들어 있습니다.
자동 검사로 잡히는 것은
일부라서 눈으로도 봅니다.

검사 WAVE — 웹 접근성 검사 도구

참고 Lighthouse — 크롬 개발자 도구에 들어 있는 페이지 점검 도구

마무리로 넣어볼 세 줄

```
p { max-width: 65ch; }
/* 한 줄이 너무 길면 읽기 어렵습니다 */

a:focus-visible {
  outline: 2px solid #0b6fb8;
  outline-offset: 2px;
}

<img src="me.jpg" alt="…" width="240" height="240">
```

ch는 글자 폭 단위

숫자 0 한 글자의 폭입니다.
65ch는 읽기 좋은 줄 길이로
널리 쓰이는 값입니다.

focus-visible

Tab 키로 이동할 때만
테두리가 보입니다.
키보드로 쓰는 사용자에게
지금 어디인지 알려줍니다.

리뷰 요청에 들어가야 할 것

1	**관련 코드**	HTML과 CSS를 함께 붙입니다. 한쪽만 주면 원인을 엉뚱한 곳에서 찾습니다.
2	**보는 기준**	시맨틱, 접근성, 선택자, 반응형 중 무엇을 볼지 정합니다.
3	**지금 상태**	무엇이 잘 되고 무엇이 어색한지 적습니다.
4	**답의 형식**	고칠 곳마다 이유와 근거를 함께 달라고 합니다.

네 가지를 갖춰 물으면 답의 품질이 눈에 띄게 달라집니다. 오늘 이 형식을 그대로 써보세요.

같은 코드, 다른 요청

이렇게 물으면

일반론이 돌아옵니다

```
내 코드 리뷰해 줘

[코드]
```

아직 내가 알지 못하는 일반적인 스펙을 제안합니다.

이렇게 물으면

내 코드 이야기가 옵니다

```
아래 HTML과 CSS를 리뷰해 주세요.
기준: 시맨틱, 접근성, 반응형
지금 상태: 좁은 화면에서 카드가 겹칩니다
형식: 고칠 곳마다 이유와 근거를 함께
```

고칠 곳이 구체적으로 나옵니다.

요청이 구체적일수록 답을 검증하기도 쉬워집니다

05 · 검증

받은 답을 그대로 믿지 않습니다

문서에서 확인

제안한 태그나 속성이
실제로 있는지 MDN에서
찾아봅니다.

화면에서 확인

바꾼 뒤 브라우저에서
화면이 의도대로 되는지
직접 봅니다.

이유를 설명

왜 그렇게 고쳤는지
한 문장으로 말할 수 있어야
반영합니다.

설명할 수 없는 코드는 넣지 않습니다

지금은 동작하더라도 다음에 문제가 생겼을 때 손을 댈 수 없게 됩니다.
반영하지 않기로 한 제안과 그 이유를 남겨두면 그 자체가 좋은 기록이 됩니다.

05 · 주의

AI 답에서 자주 나오는 세 가지

01

없는 속성

그럴듯한 이름의 속성을
만들어 내는 경우가 있습니다.
브라우저에서 아무 일도
일어나지 않습니다.

02

오래된 방식

float으로 배치하거나
table로 화면을 나누는
예전 방식을 권하기도
합니다.

03

과한 제안

지금 필요 없는 것까지
덧붙입니다. 코드가 늘면
나중에 손대기
어려워집니다.

셋 다 문서에서 한 번 확인하면 걸러집니다

속성 이름을 MDN에서 검색해 보고, 배운 방식과 다르면 왜 다른지 물어봅니다.

실습 03

내 페이지에 리뷰 요청하기

앞의 네 가지를 갖춰 AI에게 리뷰를 요청합니다.
받은 제안을 표로 옮겨 적고, 반영 여부를 스스로 정합니다.

```
아래 HTML과 CSS를 리뷰해 주세요.

기준: 시맨틱 구조, 접근성, 선택자 중복, 반응형
지금 상태: 데스크톱은 괜찮은데 좁은 화면에서 카드가 겹칩니다.
형식: 고칠 곳마다 이유와 근거 문서를 함께 적어 주세요.

[여기에 코드 전체를 붙입니다]
```

관련된 HTML과 CSS를 함께 붙입니다

한쪽만 주면 원인을 엉뚱한 곳에서 찾습니다.

정리할 표

- 제안 내용
- 근거를 확인했는가
- 반영 여부
- 그렇게 정한 이유

다 반영하지 않습니다

근거를 확인하고
설명할 수 있는 것만
반영합니다.

실습 03 · 예시

이런 표를 만듭니다

제안	근거 확인	반영	이유
section에 제목을 넣으세요	검사기 경고	반영	제목이 없으면 영역 이름이 없습니다
이미지에 width·height 지정	MDN 확인	반영	로딩 중 화면이 밀리지 않습니다
글줄 폭을 제한하세요	직접 확인	반영	넓은 화면에서 읽기 편해집니다
div를 section으로 바꾸세요	MDN 확인	보류	의미가 없는 배치용 묶음입니다
float으로 배치하세요	MDN 확인	미반영	지금은 Flexbox를 씁니다

다섯 개 중 세 개만 반영했습니다 — 반영하지 않은 이유를 적는 칸이 핵심입니다

05 · 순서

고칠 때는 이 순서로

01

구조부터

태그를 바꾸면 선택자도
따라 바뀝니다.
구조를 먼저 확정합니다.

02

그다음 스타일

선택자와 배치를 정리합니다.
안 쓰는 규칙을 지웁니다.

03

마지막에 반응형

세 폭에서 확인합니다.
분기점을 조정합니다.

한 번에 하나씩 고치고, 고칠 때마다 화면을 확인합니다

실습 03

개인 소개 페이지 완성하기

점검과 리뷰에서 나온 것을 반영해 페이지를 마무리합니다.
고친 내용과 고치지 않기로 한 내용을 함께 정리합니다.

완성 기준

- 시맨틱 태그로 영역이 나뉘어 있다
- 제목 계층과 alt가 정리돼 있다
- 선택자 충돌이 남아 있지 않다
- 세 폭에서 배치가 자연스럽다
- 검사기 오류가 0이다

정리해 둘 것

- 완성한 HTML과 CSS 파일
- 세 폭에서 찍은 화면 캡처 3장
- 리뷰 정리 표
- 반영하지 않은 제안과 그 이유
- 고친 곳 한 줄 요약

여유가 있으면 Lighthouse의 Accessibility 항목만 돌려 봅니다

다음 시간 전에 해둘 것

1	**GitHub 계정**	다음 시간에 바로 씁니다. 이메일 인증까지 마쳐 두세요
2	**파일 한 폴더에**	about-me.html, style.css, me.jpg를 한 폴더에 모아 둡니다
3	**폴더 이름**	한글과 띄어쓰기가 없는 이름으로 바꿔 둡니다. Git에서 다루기 편합니다

다음 챕터부터는 오늘 완성한 페이지를 Git으로 관리합니다

WRAP UP

오늘 정리

개념

- 반응형은 하나의 코드로 여러 화면에 대응한다
- viewport 메타 태그가 있어야 기기 폭이 기준
- min-width는 그 폭 이상일 때 적용된다
- 미디어 쿼리도 순서에 따라 덮어쓴다
- 선택자 충돌은 개발자 도구 취소선으로 보인다
- AI의 답은 근거를 확인한 것만 반영한다

직접 쓸 수 있어야 하는 것

- px을 rem으로 바꾸기
- 미디어 쿼리로 열 수 바꾸기
- 모바일 퍼스트로 규칙 쌓기
- 세 폭에서 레이아웃 오류 찾기
- 기준을 갖춰 AI 리뷰 요청하기
- 반영 여부를 판단해 페이지 완성하기

NEXT

DAY 07

Git & GitHub 개념 및 협업 실습

오늘 완성한 페이지를 이제 기록으로 남깁니다.
다음 시간에는 Git으로 변경 이력을 관리하고
GitHub에 올려 다른 사람과 함께 보는 방법을 배웁니다.

수고하셨습니다.